МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Т.В. Бессонов
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

В этой лабораторной работе вы познакомитесь с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научитесь работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Вариант: 2

Метод решения

Математическая часть

Представление вершин. Каждая вершина фигуры хранит позицию (x, y, z) и цвет (r, g, b). Для применения преобразований позиция дополняется однородной координатой w = 1, поэтому все аффинные преобразования описываются матрицами 4×4.

Построение октаэдра. Октаэдр задаётся шестью уникальными вершинами, лежащими на осях координат на расстоянии 1 от центра: (±1, 0, 0), (0, ±1, 0), (0, 0, ±1). Каждая грань — треугольник, соединяющий три вершины с разными осями. Всего получается 8 граней, которые задаются 24 индексами в буфере индексов. Использование индексного буфера позволяет не дублировать вершины: одни и те же 6 вершин переиспользуются всеми гранями.

Аффинные преобразования. Для каждой фигуры хранятся позиция центра, масштаб и углы Эйлера. Матрица переноса T строится функцией *glm::translate*, матрица масштабирования S — функцией *glm::scal*e, матрица поворота R — последовательным применением *glm::rotate* вокруг осей X, Y и Z, то есть R = Rz · Ry · Rx. В вершинном шейдере поворот применяется к локальным координатам вершины, после чего используется матрица P · T · S. Итоговое преобразование имеет вид P · T · S · R · p, где p — исходная позиция вершины, а P — матрица проекции. Локальные координаты после поворота (R · p) дополнительно применяются для процедурного задания цвета.

Процедурный цвет. Цвет вершины вычисляется в вершинном шейдере как *abs(normalize(local_coords.xyz))*, где *local_coords* — координаты вершины после поворота. Затем цвет умножается на общий цвет фигуры *main_color*, который передаётся через uniform-буфер и может изменяться

из интерфейса ImGui. Такой подход позволяет получать цвет, зависящий от положения вершины в локальной системе координат фигуры.

Перспективная проекция. Используется *glm::perspectiveFov* с вертикальным углом обзора FOV (в радианах), отношением сторон окна width / height и плоскостями отсечения near = 0.1 и far = 100.

Ортографическая проекция. Используется glm::orthoZO с симметричными границами [-half_width, half_width] и [-half_height, half_height], где half_height = 10, a half_width = half_height · aspect. Ортографическая проекция не зависит от глубины: размер объекта на экране остаётся постоянным, а параллельные прямые остаются параллельными. Диапазон глубины в Vulkan — [0, 1], поэтому применяется вариант orthoZO.

Переход к экрану. После умножения на матрицу проекции координаты попадают в нормализованное пространство устройства (NDC) Vulkan, в котором ось Y направлена вниз, а глубина изменяется от 0 до 1.

Анимация. Положение фигуры интерполируется по квадратичной кривой Безье с тремя контрольными точками: $B(t) = (1 - t)^2 \cdot P1 + 2t(1 - t) \cdot P2 + t^2 \cdot P3$. Углы Эйлера интерполируются линейно между начальным и конечным значениями. Прогресс t увеличивается на dt · speed за кадр, где dt — время между кадрами. Поддерживаются пауза, остановка и зацикливание анимации.

Объекты Vulkan

- **VkInstance, VkPhysicalDevice, VkDevice, VkQueue** — инициализация Vulkan, выбор физического устройства и создание логического устройства с графической очередью.
- **VkSurfaceKHR, VkSwapchainKHR, VkImageView** — поверхность окна, цепочка изображений для вывода на экран и представления этих изображений.
- **Изображение глубины** — создаётся через VMA в формате VK_FORMAT_D24_UNORM_S8_UINT (если он поддерживается), используется как depth-stencil вложение.
- **VkRenderPass и VkFramebuffer** — проход отрисовки с цветовым и глубинным вложениями; для каждого изображения swapchain

создаётся отдельный framebuffer. Цвет и глубина очищаются в начале каждого кадра.

- **VkCommandPool и VkCommandBuffer** — пул и командный буфер, в который каждый кадр записываются команды отрисовки сцены.
- **Синхронизация** — семафор доступности изображения, семафоры завершения отрисовки для каждого изображения swapchain и fence.
- **Графический конвейер** — два шейдерных этапа (вершинный и фрагментный); входные вершины описываются привязкой с шагом sizeof(Vertex) и двумя атрибутами (позиция и цвет); топология TRIANGLE_LIST; заливка полигонов, отсечение задних граней отключено; один сэмпл; тест и запись глубины с операцией VK_COMPARE_OP_LESS; смешивание отключено, записываются все каналы RGBA; viewport и scissor задаются динамически.
- **VkPipelineLayout и VkDescriptorSetLayout** — layout конвейера с одним дескрипторным набором (set 0, binding 0) типа uniform buffer, доступным в вершинном и фрагментном шейдерах.
- **VkDescriptorPool и VkDescriptorSet** — пул дескрипторов и отдельный дескрипторный набор для каждой фигуры. Это позволяет отрисовать несколько фигур, переключая дескрипторные наборы между вызовами отрисовки, как требуется в задании.
- **Буферы через VMA** — вершинный, индексный и uniform-буферы создаются с помощью библиотеки Vulkan Memory Allocator и отображаются в память хоста; данные копируются напрямую в отображённую память.
- **vkCmdBindVertexBuffers, vkCmdBindIndexBuffer, vkCmdBindDescriptorSets, vkCmdDrawIndexed** — привязка ресурсов и отрисовка 8 треугольников по 24 индексам для каждой фигуры.
- **ImGui** — интерфейс отрисовывается отдельным командным буфером и отдельным render pass в том же кадре; через него настраиваются параметры фигур, проекция, цвет и анимация.

Результаты

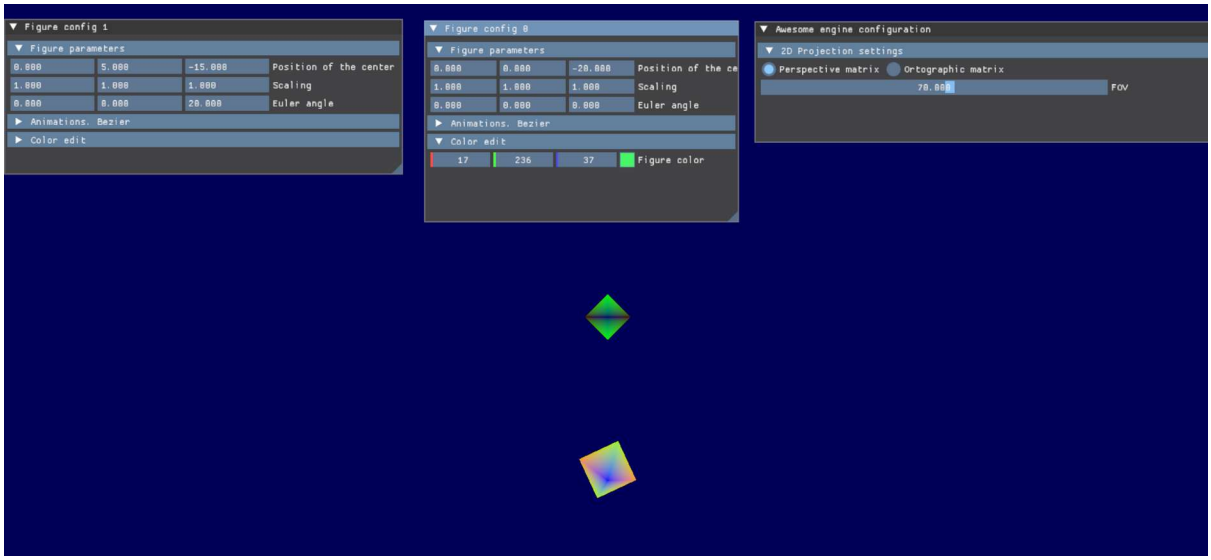


Рисунок 1. 2 октаэдра с разной ориентацией, расстоянием до камеры и цветом. Включена перспективная проекция.

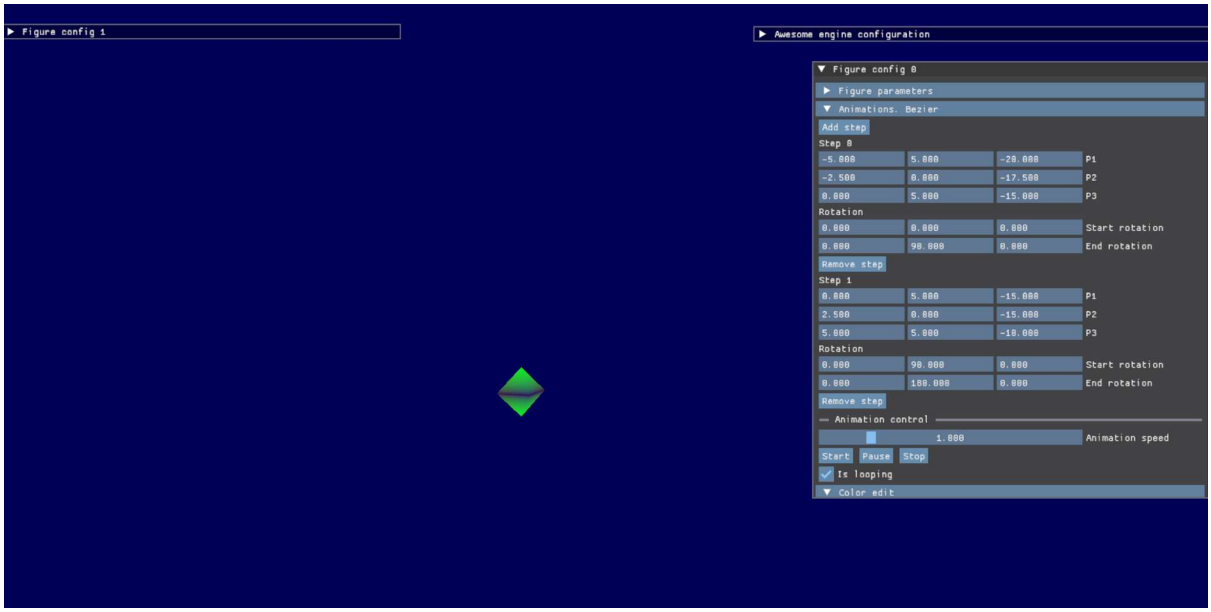


Рисунок 2. Для одного из октаэдров запущена анимация. На ImGui окне показаны параметры анимации: точки для квадратичной кривой Безье и ориентация фигуры.

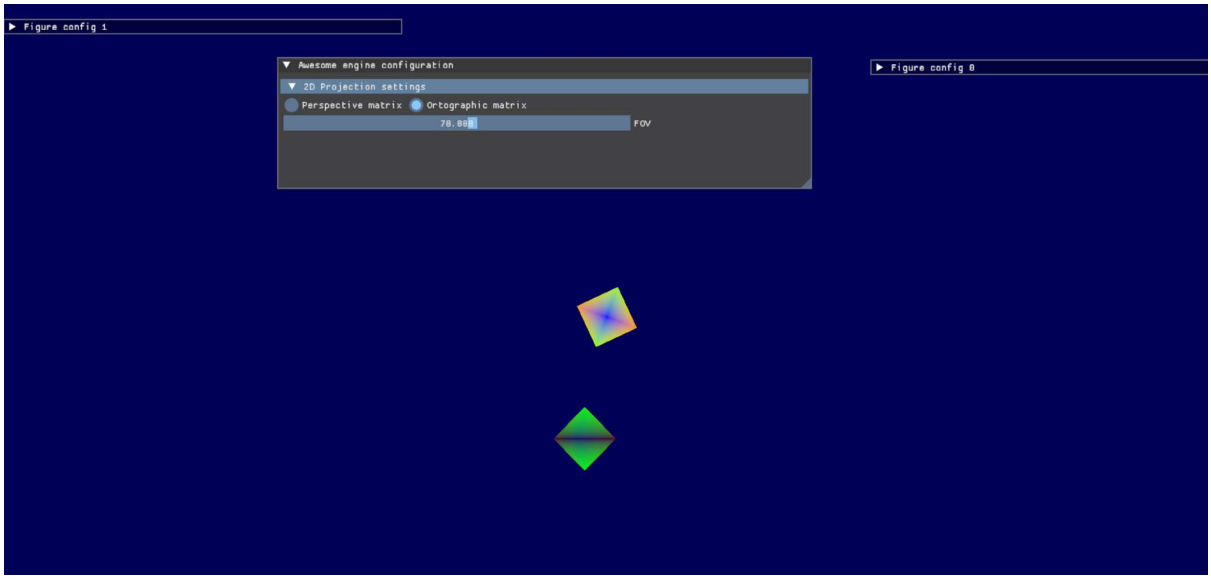


Рисунок 3. Ортографическая проекция

Выводы

Я научился строить простой 3D-объект (октаэдр) с помощью вершинного и индексного буферов, применять аффинные преобразования (перенос, масштабирование и поворот) и строить матрицы перспективной и ортографической проекции. Я освоил передачу матриц и параметров в шейдеры через uniform-буферы и дескрипторные наборы, настройку графического конвейера Vulkan, организацию покадровой отрисовки нескольких фигур с отдельными дескрипторными наборами, а также анимацию объектов с помощью кривых Безье.